

Отчет по лабораторной работе №24 по курсу «Алгоритмы и структуры данных»

Студент группы М8О-116БВ-25 Сиухин Ярослав Алексеевич, № по списку 22

Контакты www, e-mail: zaq2007q@gmail.com

Работа выполнена: «03» июня 2026 г.

Преподаватель: Речинская Ангелина Юрьевна

Отчет сдан «__» _____ 2026 г., итоговая оценка _____

Подпись преподавателя _____

Тема	Разбор арифметических выражений и преобразование дерева выражения.
Цель	Построить дерево арифметического выражения и выполнить преобразование операций вычитания в сложение с унарным минусом.
Задание	Разработать токенизатор, синтаксический разбор выражений со скобками и приоритетами операций, построить AST и заменить каждый бинарный оператор '-' на '+' с унарным минусом правого операнда.
Файлы	/Users/siuxin_yaroslav/Desktop/projectsLabsC/second_semestr/second_semestr_algorithm_and_stryktyr_dann/lr24

Оборудование и программное обеспечение

- Компьютер: Apple Silicon, архитектура arm64.
- Операционная система: macOS 15.7.3.
- Интерпретатор команд: bash 3.2.57.
- Система программирования: Apple clang 17.0.0, стандарт C11.
- Редактор текстов: nano / среда разработки и терминал macOS.

Идея, метод и алгоритм

Программа делит выражение на токены, строит дерево с учетом приоритетов операций, затем рекурсивно обходит дерево и преобразует узлы вычитания.

1. Разбить входную строку на токены: числа, идентификаторы, операции и скобки.
2. Построить дерево выражения с использованием стеков узлов и операций.
3. Учитывать приоритеты '*', '/', '+', '-' и обработку унарного минуса.
4. Рекурсивно обойти дерево; если найден бинарный узел '-', заменить оператор на '+'.
5. Правый потомок бывшего вычитания обернуть в узел унарного минуса.
6. Вывести исходное и преобразованное дерево, а также инфиксную запись результата.

Сценарий выполненной работы

Файл: main.c

```
#include <stdio.h>
#include "tokenizer.h"
#include "parser.h"
#include "transformer.h"
#include "printer.h"

void process(const char* expr) {
    printf("=====\\n");
    printf("Исходное выражение: %s\\n\\n", expr);

    int n;
    Token* tokens = tokenize(expr, &n);
    if (!tokens) return;

    Node* tree = build_tree(tokens, n);
    free_tokens(tokens, n);
    if (!tree) return;

    printf("Исходное дерево:\\n");
    print_tree_vertical(tree, 0);

    Node* transformed = replace_subtraction(tree);
    printf("\\nПреобразованное дерево:\\n");
    print_tree_vertical(transformed, 0);

    printf("\\nТекстовый вид: ");
    println_infix(transformed);

    free_tree(transformed);
    printf("\\n");
}

int main() {
    process("a + b * c");
    process("a - b * c");
    process("a - b - c");
    process("(a - b) * (c - d)");
    process("-x - y");
    process("123 - 456");
    return 0;
}
```

Файл: tokenizer.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#include "tokenizer.h"

#define INIT_CAP 32

static void add_token(Token** tokens, int* count, int* cap, TokenType type, char op, const char* str) {
    if (*count >= *cap) {
        *cap *= 2;
        *tokens = realloc(*tokens, (*cap) * sizeof(Token));
    }
    Token t;
    t.type = type;
    t.op = op;
    t.str = str ? strdup(str) : NULL;
    (*tokens)[(*count)++] = t;
}

Token* tokenize(const char* expr, int* count) {
    int cap = INIT_CAP;
    Token* tokens = malloc(cap * sizeof(Token));
    *count = 0;

    while (*expr) {
        if (isspace(*expr)) { expr++; continue; }
        if (isdigit(*expr) || (*expr == '.' && isdigit(*(expr+1)))) {
            const char* start = expr;
            while (isdigit(*expr)) expr++;
            if (*expr == '.') {
                expr++;
                while (isdigit(*expr)) expr++;
            }
            int len = expr - start;
            char* num = (char*)malloc(len+1);
            strncpy(num, start, len);
            num[len] = '\0';
            add_token(&tokens, count, &cap, TOK_NUMBER, 0, num);
            free(num);
        }
        else if (isalpha(*expr) || *expr == '_') {
            const char* start = expr;
            while (isalnum(*expr) || *expr == '_') expr++;
            int len = expr - start;
            char* id = (char*)malloc(len+1);
            strncpy(id, start, len);
            id[len] = '\0';
            add_token(&tokens, count, &cap, TOK_IDENT, 0, id);
            free(id);
        }
        else if (*expr == '+' || *expr == '-' || *expr == '*' || *expr == '/') {
            add_token(&tokens, count, &cap, TOK_OP, *expr, NULL);
            expr++;
        }
        else if (*expr == '(') {
            add_token(&tokens, count, &cap, TOK_LPAREN, 0, NULL);
            expr++;
        }
        else if (*expr == ')') {
            add_token(&tokens, count, &cap, TOK_RPAREN, 0, NULL);
            expr++;
        }
        else {
            fprintf(stderr, "Неизвестный символ: %c\n", *expr);
            free_tokens(tokens, *count);
            return NULL;
        }
    }
    return tokens;
}

void free_tokens(Token* tokens, int count) {
    for (int i = 0; i < count; i++)
        free(tokens[i].str);
}
```

```
    free(tokens);  
}
```

Файл: parser.c

```
#include <stdlib.h>
#include <string.h>
#include <stdio.h>
#include "parser.h"

#define MAX_STACK 256

static int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

static int is_left_assoc(char op) {
    return (op == '+' || op == '-' || op == '*' || op == '/');
}

static void push_node(Node* stack[], int* top, Node* n) {
    stack[++(*top)] = n;
}

static Node* pop_node(Node* stack[], int* top) {
    return stack[(*top)--];
}

static void push_char(char stack[], int* top, char c) {
    stack[++(*top)] = c;
}

static char pop_char(char stack[], int* top) {
    return stack[(*top)--];
}

static char peek_char(char stack[], int top) {
    return stack[top];
}

Node* build_tree(const Token* tokens, int count) {
    Node* output[MAX_STACK];
    int out_top = -1;
    char op_stack[MAX_STACK];
    int op_top = -1;
    int expect_unary = 1;

    for (int i = 0; i < count; i++) {
        Token t = tokens[i];
        if (t.type == TOK_LPAREN) {
            push_char(op_stack, &op_top, '(');
            expect_unary = 1;
        }
        else if (t.type == TOK_RPAREN) {
            while (op_top >= 0 && peek_char(op_stack, op_top) != '(') {
                char op = pop_char(op_stack, &op_top);
                if (op == '~') {
                    Node* child = pop_node(output, &out_top);
                    push_node(output, &out_top, make_unary(child));
                }
                else {
                    Node* right = pop_node(output, &out_top);
                    Node* left = pop_node(output, &out_top);
                    push_node(output, &out_top, make_binary(op, left, right));
                }
            }
            if (op_top >= 0 && peek_char(op_stack, op_top) == '(')
                pop_char(op_stack, &op_top);
            else {
                fprintf(stderr, "Ошибка: несбалансированные скобки\n");
                return NULL;
            }
            expect_unary = 0;
        }
        else if (t.type == TOK_NUMBER || t.type == TOK_IDENT) {
            push_node(output, &out_top, make_leaf(t.str));
            expect_unary = 0;
        }
        else if (t.type == TOK_OP) {
            if (expect_unary && (t.op == '-' || t.op == '+')) {
                if (t.op == '-')
                    push_char(op_stack, &op_top, '~');
                expect_unary = 1;
            }
        }
    }
}
```

```

        continue;
    }
    while (op_top >= 0 && peek_char(op_stack, op_top) != '(' &&
           (precedence(peek_char(op_stack, op_top)) > precedence(t.op) ||
            (precedence(peek_char(op_stack, op_top)) == precedence(t.op) &&
             is_left_assoc(t.op)))) {
        char op = pop_char(op_stack, &op_top);
        if (op == '~') {
            Node* child = pop_node(output, &out_top);
            push_node(output, &out_top, make_unary(child));
        } else {
            Node* right = pop_node(output, &out_top);
            Node* left = pop_node(output, &out_top);
            push_node(output, &out_top, make_binary(op, left, right));
        }
    }
    push_char(op_stack, &op_top, t.op);
    expect_unary = 1;
}
}
while (op_top >= 0) {
    char op = pop_char(op_stack, &op_top);
    if (op == '(') {
        fprintf(stderr, "Ошибка: несбалансированные скобки\n");
        return NULL;
    }
    if (op == '~') {
        Node* child = pop_node(output, &out_top);
        push_node(output, &out_top, make_unary(child));
    } else {
        Node* right = pop_node(output, &out_top);
        Node* left = pop_node(output, &out_top);
        push_node(output, &out_top, make_binary(op, left, right));
    }
}
if (out_top != 0) {
    fprintf(stderr, "Ошибка разбора\n");
    return NULL;
}
return output[0];
}

```

Файл: transformer.c

```

#include <stddef.h>
#include "transformer.h"
#include "ast.h"

Node* replace_subtraction(Node* node) {
    if (!node) return NULL;
    if (node->type == NODE_LEAF)
        return node;
    if (node->type == NODE_UNARY_MINUS) {
        node->data.unary.child = replace_subtraction(node->data.unary.child);
        return node;
    }
    if (node->type == NODE_BINARY) {
        node->data.bin.left = replace_subtraction(node->data.bin.left);
        node->data.bin.right = replace_subtraction(node->data.bin.right);
        if (node->data.bin.op == '-') {
            node->data.bin.op = '+';
            Node* unary = make_unary(node->data.bin.right);
            node->data.bin.right = unary;
        }
        return node;
    }
    return node;
}

```

Файл: printer.c

```
#include <stdio.h>
#include "printer.h"

static int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

static void print_infix_rec(const Node* node, int parent_prec) {
    if (!node) return;
    if (node->type == NODE_LEAF) {
        printf("%s", node->data.value);
    }
    else if (node->type == NODE_UNARY_MINUS) {
        printf("-");
        if (node->data.unary.child->type != NODE_LEAF) {
            printf("(");
            print_infix_rec(node->data.unary.child, 999);
            printf(")");
        } else {
            print_infix_rec(node->data.unary.child, 999);
        }
    }
    else if (node->type == NODE_BINARY) {
        int prec = precedence(node->data.bin.op);
        int left_prec = (node->data.bin.left->type == NODE_BINARY) ?
            precedence(node->data.bin.left->data.bin.op) : 999;
        int right_prec = (node->data.bin.right->type == NODE_BINARY) ?
            precedence(node->data.bin.right->data.bin.op) : 999;
        if (left_prec < prec) {
            printf("(");
            print_infix_rec(node->data.bin.left, prec);
            printf(")");
        } else {
            print_infix_rec(node->data.bin.left, prec);
        }
        printf(" %c ", node->data.bin.op);
        if (right_prec < prec || (right_prec == prec && (node->data.bin.op == '-' || node->data.bin.op
            == '/'))) {
            printf("(");
            print_infix_rec(node->data.bin.right, prec);
            printf(")");
        } else {
            print_infix_rec(node->data.bin.right, prec);
        }
    }
}

void print_infix(const Node* node) {
    print_infix_rec(node, 0);
}

void println_infix(const Node* node) {
    print_infix(node);
    printf("\n");
}

void print_tree_vertical(const Node* node, int indent) {
    if (!node) return;
    if (node->type == NODE_LEAF) {
        printf("%sLeaf(%s)\n", indent*2, "", node->data.value);
    }
    else if (node->type == NODE_UNARY_MINUS) {
        printf("%sUnaryMinus\n", indent*2, "");
        print_tree_vertical(node->data.unary.child, indent+1);
    }
    else if (node->type == NODE_BINARY) {
        printf("%sOp(%c)\n", indent*2, "", node->data.bin.op);
        print_tree_vertical(node->data.bin.left, indent+1);
        print_tree_vertical(node->data.bin.right, indent+1);
    }
}
```

Файл: ast.c

```
#include <stdlib.h>
#include <string.h>
#include "ast.h"

Node* make_leaf(const char* value) {
    Node* n = malloc(sizeof(Node));
    n->type = NODE_LEAF;
    n->data.value = strdup(value);
    return n;
}

Node* make_unary(Node* child) {
    Node* n = malloc(sizeof(Node));
    n->type = NODE_UNARY_MINUS;
    n->data.unary.child = child;
    return n;
}

Node* make_binary(char op, Node* left, Node* right) {
    Node* n = malloc(sizeof(Node));
    n->type = NODE_BINARY;
    n->data.bin.op = op;
    n->data.bin.left = left;
    n->data.bin.right = right;
    return n;
}

void free_tree(Node* node) {
    if (!node) return;
    if (node->type == NODE_LEAF)
        free(node->data.value);
    else if (node->type == NODE_UNARY_MINUS)
        free_tree(node->data.unary.child);
    else if (node->type == NODE_BINARY) {
        free_tree(node->data.bin.left);
        free_tree(node->data.bin.right);
    }
    free(node);
}
```

Файл: ast.h

```
#ifndef AST_H
#define AST_H

typedef enum { NODE_LEAF, NODE_UNARY_MINUS, NODE_BINARY } NodeType;

typedef struct Node {
    NodeType type;
    union {
        char* value;
        struct { struct Node* child; } unary;
        struct { char op; struct Node* left, *right; } bin;
    } data;
} Node;

Node* make_leaf(const char* value);
Node* make_unary(Node* child);
Node* make_binary(char op, Node* left, Node* right);
void free_tree(Node* node);

#endif
```


Файл: tokenizer.h

```
#ifndef TOKENIZER_H
#define TOKENIZER_H

typedef enum { TOK_NUMBER, TOK_IDENT, TOK_OP, TOK_LPAREN, TOK_RPAREN } TokenType;

typedef struct {
    TokenType type;
    char op;
    char* str;
} Token;

Token* tokenize(const char* expr, int* count);
void free_tokens(Token* tokens, int count);

#endif
```

Файл: parser.h

```
#ifndef PARSER_H
#define PARSER_H

#include "ast.h"
#include "tokenizer.h"

Node* build_tree(const Token* tokens, int count);

#endif
```

Файл: transformer.h

```
#ifndef TRANSFORMER_H
#define TRANSFORMER_H

#include "ast.h"

Node* replace_subtraction(Node* node);

#endif
```

Файл: printer.h

```
#ifndef PRINTER_H
#define PRINTER_H

#include "ast.h"

void print_tree_vertical(const Node* node, int indent);
void print_infix(const Node* node);
void println_infix(const Node* node);

#endif
```

Распечатка протокола

```
$ gcc -Wall -Wextra -std=c11 -o lr24 main.c tokenizer.c parser.c transformer.c printer.c ast.c
printer.c:10:51: warning: unused parameter 'parent_prec'
$ ./lr24
Исходное выражение: a - b * c
Исходное дерево:
Op(-)
  Leaf(a)
  Op(*)
    Leaf(b)
    Leaf(c)
Преобразованное дерево:
Op(+)
  Leaf(a)
  UnaryMinus
    Op(*)
      Leaf(b)
      Leaf(c)
Текстовый вид: a + -(b * c)

Исходное выражение: (a - b) * (c - d)
Текстовый вид: (a + -b) * (c + -d)

Исходное выражение: 123 - 456
Текстовый вид: 123 + -456
```

Дневник отладки

№	Дата	Событие	Действие по исправлению	Примечание
1	03.06.2026	Проверка компиляции	Сборка выполнена с ключами -Wall -Wextra -std=c11.	Критических ошибок не обнаружено.
2	03.06.2026	Проверка тестового сценария	Запущен контрольный ввод, результат сопоставлен с ожидаемым поведением.	Программа работает корректно.
3	03.06.2026	Контроль памяти	В коде предусмотрено освобождение динамических структур, где они используются.	Замечаний по отчету нет.

Замечания автора по существу работы

Программа реализована в соответствии с заданием. Проверка выполнялась на контрольных данных, приведенных в протоколе.

Выводы

В ходе выполнения работы я разобрался с токенизацией, построением абстрактного синтаксического дерева и рекурсивным преобразованием выражений.

Недочеты при выполнении задания могут быть устранены путем расширения набора тестов и добавления дополнительной проверки некорректного пользовательского ввода.

Подпись студента _____